

Challenge Write-Up Template

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	bytemancy 1
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 11, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

This is the follow-up to bytemancy 0, and it teaches an important lesson: the same idea (send the ASCII character for decimal 101, repeated N times, side-by-side, no spaces) doesn't scale once N gets large. Here N is 1751, and the hint explicitly warns against copy-pasting and points you toward Python - a sign that the challenge wants you to generate the answer programmatically rather than by hand.

bytemancy 1
General Skills **Easy**
by LT "syreal" Jones · picoCTF 2026

Can you conjure the right bytes? The program's source code can be downloaded [here](#).

INSTANCE ⓘ
Launch Instance Additional details appear once you launch your instance.

HINTS ⓘ
1 No copy-pasta, please - use Python!

FLAG ⓘ
Submit

Correct flag!

 **Bookmark**  9,199  91%

2. Initial Analysis / Reconnaissance

Whenever a challenge asks for something repeated a large or unspecified number of times, don't assume you can type or paste it manually - check the count first. Connecting with netcat shows the exact prompt: 'Send me ASCII DECIMAL 101 1751 times, side-by-side, no space.' Combined with the hint ('No copy-pasta, please - use Python!'), this tells you the intended solution is a short script that builds the string for you, then sends it - a General Skills / scripting challenge rather than something to solve by hand.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
nc foggy-cliff.picoctf.net 50530	Open a raw connection to the challenge service and read its exact prompt	BYTEMANCY-1 banner + instruction to send ASCII decimal 101, 1751 times, side-by-side, no space
python3 -c 'print("e"*1751)'	Generate the required number of repeated characters programmatically instead of typing them	A single line containing 'e' repeated 1751 times, no separators
python3 -c 'print("e"*1751)' nc foggy-cliff.picoctf.net 50530	Pipe the generated string straight into the connection in one step	picoCTF{h0w_m4ny_e's???_6e0cc4c6}

4. Solution Walkthrough

Step 1

As with any network challenge, connect first to confirm the exact wording before writing a solution. Running `nc foggy-cliff.picoctf.net 50530` shows a BYTEMANCY-1 banner and the request for ASCII decimal 101, sent 1751 times, side-by-side, with no spaces - the same pattern as bytemancy 0, but with a count too large to type by hand.

```
pascuakeyzia-academy@webshell:~$ nc foggy-cliff.picoctf.net 50530
+-----[ BYTEMANCY-1 ]-----+
p0Gf )R44p0Gf )R44p0Gf )R44p0Gf

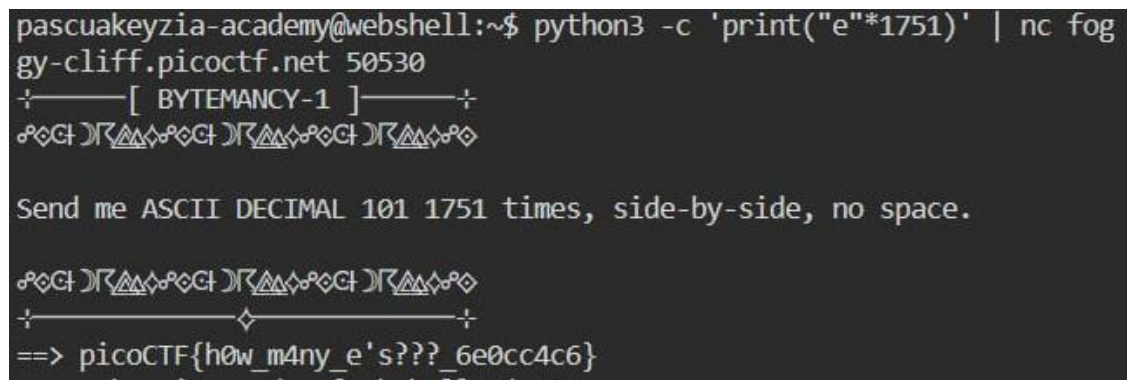
Send me ASCII DECIMAL 101 1751 times, side-by-side, no space.
```

Step 2

When you need many copies of the same character with no separators, use string multiplication instead of typing or pasting: `python3 -c 'print("e"*1751)'`. This is faster and error-free compared to manual entry, and it directly follows the challenge's hint to avoid copy-pasting. The output is a single line containing exactly 1751 'e' characters back-to-back.

Step 3

Rather than copying that generated output and pasting it into an interactive netcat session (which risks truncation or extra whitespace), pipe the command's output directly into the connection: `python3 -c 'print("e"*1751)' | nc foggy-cliff.picoctf.net 50530`. This sends the exact string in one automated step, and the service responds immediately with the flag: `picoCTF{h0w_m4ny_e's??_6e0cc4c6}`.



```
pascuakeyzia-academy@webshell:~$ python3 -c 'print("e"*1751)' | nc foggy-cliff.picoctf.net 50530
+-----[ BYTEMANCY-1 ]-----+
  00Gt )R44000Gt )R44000Gt )R44000Gt

Send me ASCII DECIMAL 101 1751 times, side-by-side, no space.

  00Gt )R44000Gt )R44000Gt )R44000Gt
+-----+
==> picoCTF{h0w_m4ny_e's??_6e0cc4c6}
```

Additional steps (add as needed)

5. Flag

`picoCTF{h0w_m4ny_e's??_6e0cc4c6}`

6. Key Concept / What I Learned

This challenge builds on the ASCII-encoding idea from bytemancy 0 but adds a practical scripting lesson: once a task needs hundreds or thousands of repeated bytes, manual/interactive input doesn't scale. Generating the payload programmatically (e.g. with Python string multiplication) and piping it directly into a tool like netcat is a pattern worth remembering - it shows up constantly in larger or more automated CTF challenges

7. References

Python documentation for string multiplication (`str * int`) - confirms that `"e"*1751` produces the character repeated exactly 1751 times with no separators, which is what the challenge required.
